

Capitolo 2a - Servizi dei SO

Status Done

Un SO offre un ambiente in cui eseguire i programmi e fornire servizi. I servizi variano a seconda del SO:

- **Interfaccia con l'utente:** quasi tutti i SO hanno un'interfaccia con l'utente (UI). La forma più diffusa è l'interfaccia utente grafica (GUI), ossia un sistema a finestre dotato di un dispositivo puntatore (es. mouse). I dispositivi mobili offrono un'interfaccia **touch screen**, mentre un'altra possibilità è l'interfaccia a **riga di comando**.
- **Esecuzione di un programma:** il sistema deve poter caricare un programma in memoria ed eseguirlo.
- **Operazioni di I/O:** un programma di esecuzione può richiedere un'operazione di I/O che implica l'uso di un file o di un dispositivo di I/O.
- **Gestione del file system:** il file system riveste un interesse particolare. I programmi richiedono l'esecuzione di operazioni di lettura e scrittura su file, oltre a creare e cancellare directory. Alcuni SO, infine, gestiscono i permessi di accesso ai file sulla base della proprietà del file interessato.
- **Comunicazioni:** in molti casi un processo ha bisogno di scambiare informazioni con un altro processo. La comunicazione si può realizzare tramite una memoria condivisa, che permette a due o più processi di leggere e scrivere in una porzione di memoria che condividono, o attraverso lo scambio di messaggi, in questo caso il sistema operativo trasferisce pacchetti d'informazione.
- **Rilevamento di errori:** il SO deve essere sempre capace di rilevare e correggere eventuali errori che possono verificarsi nella cpu e nei dispositivi di memoria, di I/O e in un programma utente. Per assicurare un'elaborazione corretta e coerente il SO deve saper intraprendere l'azione giusta per ciascun tipo d'errore. Talvolta l'unica scelta possibile è l'arresto del sistema, altre volte è possibile solamente terminare il processo.
- **Allocazione delle risorse:** se sono attivi più utenti o sono contemporaneamente in esecuzione più processi, il SO provvede all'assegnazione delle risorse necessarie a ciascuno di essi.
- **Logging:** vogliamo mantenere traccia di quali programmi usano il calcolatore, segnando quali e quante risorse impiegano.
- **Protezione e sicurezza:** i proprietari di informazioni memorizzate in un sistema elaborativo multiutente o in rete possono voler controllare l'uso di tali informazioni. La protezione assicura che l'accesso alle risorse del sistema sia controllato. La sicurezza di un sistema comincia con la richiesta d'identificazione da parte di ciascun utente.

Interfaccia con l'utente del SO

Vi sono due modi fondamentali per gli utenti di comunicare con il SO:

- **Interprete dei comandi:** permette agli utenti di inserire direttamente le istruzioni da eseguire.
- **Interfaccia grafica:** serve da tramite fra l'utente e il sistema.

Interprete dei comandi (Shell)

L'interprete dei comandi è un programma speciale che si avvia all'accesso dell'utente (login) o all'inizio di un processo.

Quando un sistema (es. Unix, Linux) offre la scelta tra più interpreti, questi sono chiamati **shell**.

La funzione principale è raccogliere ed eseguire i comandi dell'utente (spesso per la gestione di file).

I comandi possono essere implementati in due modi:

- **Codice interno alla shell:** l'interprete contiene il codice per eseguire il comando (dimensione della shell proporzionale al numero di comandi).
- **Programmi di sistema separati (Unix):** la maggior parte dei comandi è implementata come file eseguibili separati (`rm file.txt` cerca ed esegue il file `rm`). Questo rende l'interprete dei comandi più piccolo e permette di aggiungere facilmente nuovi comandi creando nuovi file.

Interfaccia Grafica con l'Utente (GUI)

- La **GUI** (Graphical User Interface) è un modo di comunicare con il sistema più **user-friendly**.
- Invece di digitare comandi, gli utenti interagiscono con **finestre, menu, icone e puntatore del mouse** (metafora della **scrivania** o *desktop*).
- Storicamente introdotta da Xerox PARC (primo computer Xerox Alto 1973), ma **resa popolare** da **Apple Macintosh** e poi da **Microsoft Windows**.
- Sistemi come Unix/Linux offrono anche GUI come **KDE** e **GNOME**, spesso open-source.

Interfaccia Touch-Screen

- Utilizzata principalmente da **sistemi mobili (smartphone e tablet)**, dove tastiera e mouse non sono pratici.
- L'interazione avviene tramite **gesti (gesture)**, come premere e scorrere le dita direttamente sullo schermo.
- Simula la tastiera sul display (es. l'interfaccia **Springboard** per iPhone e iPad).

Scelta e Preferenze

- La scelta dell'interfaccia è spesso una questione di **preferenze personali e tipo di utente**.
- Gli **amministratori di sistema** e gli **utenti esperti** preferiscono le **interfacce a riga di comando (CLI)** perché sono generalmente **più efficienti**, consentono un accesso più veloce alle funzionalità avanzate e sono **programmabili** (attraverso gli **shell script**) per automatizzare task ripetuti.
- Molti sistemi espongono le **funzionalità meno comuni** solo tramite riga di comando.
- Sistemi moderni (come **macOS** - che include il kernel Unix - e **Windows**) spesso offrono **entrambe** le interfacce (**GUI e CLI**).
- Sui sistemi mobili, l'interfaccia **touch-screen** è quasi l'unica modalità di interazione.

La progettazione dell'interfaccia è generalmente **distinta dalla struttura fondamentale** del sistema operativo.

Chiamate di sistema

Le **system call** formano un'interfaccia per i servizi resi disponibili dal SO, generalmente sono sotto forma di routine scritte in c o c++, per alcuni compiti di basso livello è necessario assembly.

Interfaccia per la programmazione di applicazioni (API)

I programmi hanno bisogno di interagire costantemente con il SO per eseguire operazioni essenziali (es. leggere un file, creare un processo, ecc.).

Queste operazioni sono fatte tramite **system calls**, e un sistema può eseguirne migliaia al secondo.

Le chiamate di sistema sono spesso **molto dettagliate**, complesse da usare e i loro nomi cambiano da un SO all'altro. Per semplificare il lavoro del programmatore, si utilizza un'API (Application Programming Interface).

L'API è un **insieme di funzioni standardizzate** che i programmatori di applicazioni usano. Un programmatore non invoca direttamente la chiamata di sistema, ma usa la funzione API.

Esempi:

- API di Windows
- API POSIX (per sistemi basati su standard Unix, Linux, macOS)
- API Java (per applicazioni eseguite sulla JVM)

Vantaggi API:

- **Portabilità:** un programma scritto usando un'API standard può essere compilato ed eseguito su qualsiasi sistema che supporti quella stessa API, anche se i SO sottostanti sono diversi.
- **Semplicità:** le funzioni API sono generalmente più semplici e meno dettagliate rispetto alle dirette chiamate di sistema.
- **Astrattismo:** l'API nasconde i complessi dettagli di implementazione della chiamata di sistema del kernel.

Quando un programma esegue una funzione API:

1. l'applicazione invoca la funzione API (contenuta in una libreria di codice come **libc** in Unix/Linux);
2. la funzione API viene intercettata dall'Ambiente di Esecuzione al Run-Time (RTE);
3. l'RTE, attraverso l'interfaccia alle chiamate di sistema (*System Call Interface*), converte la richiesta API nella corrispondente chiamata di sistema numerica;
4. la richiesta numerica viene inviata al kernel del SO;
5. il kernel esegue l'operazione richiesta e restituisce lo stato o i dati.

Per fornire alla chiamata di sistema le informazioni necessarie:

- **Registri:** i parametri vengono passati direttamente nei registri della cpu (il metodo più veloce).
- **Blocco in memoria:** se i parametri sono troppi i registri vengono memorizzati in un'area di memoria (blocco o tabella), e l'indirizzo di questo blocco viene passato in un registro.
- **Stack:** i parametri vengono inseriti (push) nello stack del programma e prelevati (pop) dal SO.

Categorie di chiamate di sistema

Le chiamate di sistema sono classificabili approssimativamente in sei categorie principali:

- controllo dei processi;
- gestione dei file;
- gestione dei dispositivi;
- gestione delle informazioni;
- comunicazione;
- protezione.

Controllo dei processi

Un programma in esecuzione (un processo) deve potersi fermare:

- **terminazione normale** (`end()` / `exit()`): il programma finisce il suo lavoro senza problemi, spesso rappresentato da un codice di errori pari a 0;
- **terminazione anomala** (`abort()`): il programma si interrompe a causa di un errore (un bug, dati errati, eccezioni).
 - In caso di errore anomalo, il SO può generare un **dump** (copia del contenuto della memoria) e un messaggio di errore.
 - Questo **dump** è fondamentale: i programmatori usano un programma chiamato **debugger** per analizzare il contenuto della memoria al momento dell'errore e trovare la causa del problema.
 - La terminazione anomala può essere segnalata da un **livello d'errore** (un codice diverso da zero) che indica la gravità del problema.

Dopo che un programma termina (normalmente o anormalmente), il SO deve **restituire il controllo all'interfaccia utente**:

- **Interfaccia a riga di comando (CLI)**: L'interprete dei comandi (shell) semplicemente chiede il comando successivo, presumendo che l'utente gestirà l'errore (se presente).
- **Interfaccia grafica (GUI)**: Una finestra di dialogo avverte l'utente dell'errore e chiede istruzioni.
- **Sistema a lotti**: L'interprete abortisce il "lavoro" corrente e passa automaticamente al successivo.

Un processo (come la shell) può chiedere al SO di caricare ed eseguire un altro programma (`load()` ed `execute()`).

- Quando un programma ne avvia un altro, ci sono diverse modalità di esecuzione:
 - **Sostituzione**: il programma chiamante termina e viene sostituito dal nuovo programma.
 - **Sospensione/Attesa**: il programma chiamante si sospende (`wait()`) e attende la fine del nuovo programma. Al termine, il controllo torna al chiamante, riprendendo l'esecuzione da dove si era fermato (si salva l'immagine della memoria).
 - **Esecuzione concorrente (multitasking)**: il programma chiamante continua a funzionare contemporaneamente al nuovo programma. Questo richiede una chiamata specifica come `create_process()` o, in Unix, la combinazione di `fork()` e `exec()` .

Il SO fornisce chiamate di sistema per mantenere il controllo e coordinare i processi, specialmente in ambienti multitasking:

- **Attributi del processo**: si possono leggere e impostare le proprietà di un processo (come la priorità o il tempo massimo di esecuzione) usando chiamate come `get_process_attributes()` e `set_process_attributes()` .
- **Sincronizzazione**: i processi possono aver bisogno di aspettare che un altro processo finisca (`wait()`) o che si verifichi un evento specifico (`wait_event()`).
- **Integrità dei dati condivisi**: quando più processi condividono gli stessi dati, il SO fornisce meccanismi di blocco (*locking*) come `acquire_lock()` e `release_lock()` . Questi assicurano che solo un processo alla volta possa modificare i dati, prevenendo la corruzione.

Gestione dei file

Le chiamate di sistema relative ai file sono l'insieme di funzioni che permettono ai programmi di interagire con i dati memorizzati su disco.

Per utilizzare un file, un programma deve eseguire una sequenza logica di operazioni:

- **Creazione e cancellazione**:
 - `create()`
 - `delete()`

- **Accesso:**

- `open()` : prepara il file per l'uso, essenziale prima di leggere o scrivere;
- `close()` : rilascia il file.

Una volta che il file è aperto, si possono eseguire le operazioni sui dati effettivi:

- `read()` : legge i dati dal file e li trasferisce in un buffer nella memoria del programma.
- `write()` : scrive i dati dal programma al file.
- `reposition()` : permette di cambiare la posizione corrente all'interno del file (ad esempio, tornare all'inizio o saltare alla fine) per la successiva operazione di lettura e scrittura.

I file e le directory non sono solo dati, ma hanno anche informazioni descrittive (attributi) che devono essere gestite:

- **Attributi dei file:** includono nome, tipo (testo, eseguibile, immagine), **codici di protezione** (permessi di lettura/scrittura/esecuzione) e **informazioni di accounting** (data di creazione, ultima modifica, proprietario, ecc.).
- `get_file_attributes()` : Ottiene il valore degli attributi di un file o directory.
- `set_file_attributes()` : Permette di modificare gli attributi (ad esempio, cambiare i permessi o il proprietario).

Non tutti i sistemi operativi offrono chiamate di sistema dirette per operazioni più complesse come **spostare** (`move()`) o **copiare** (`copy()`) un file.

- Alcuni SO forniscono chiamate dedicate.
- Altri SO delegano queste operazioni a **API** che combinano chiamate di sistema più semplici (ad esempio, una `copy()` API potrebbe usare `open()`, `read()`, `write()` e `close()` internamente).
- Altri ancora offrono queste operazioni solo come **programmi di sistema** (es. i comandi `cp` o `mv` in Unix/Linux) che possono essere eseguiti da altri programmi.

Queste stesse funzioni (`create`, `delete`, `open`, `close`, `read`, `write`, `get/set attributes`) sono necessarie anche per gestire le **directory** (cartelle), che sono un tipo speciale di file che organizza gli altri file.

Gestione dei dispositivi

Un processo ha bisogno di diverse risorse per operare:

- spazio in memoria (RAM);
- spazio su disco;
- accesso a file;
- accesso a dispositivi fisici (dischi, stampanti, schede di rete, ecc.).

Se queste risorse sono disponibili, il SO le assegna al processo; se non lo sono, il processo deve attendere finché non vengono rilasciate da altri.

Il SO tratta molte risorse come dei dispositivi:

- **dispositivi fisici:** componenti hardware reali;
- **dispositivi astratti/virtuali:** entità logiche (es. file).

Per garantire un uso ordinato e, a volte, esclusivo delle risorse, i SO utilizzano chiamate di sistema di "blocco":

Operazione	Chiamata di Sistema	Analogo con i File	Scopo
Richiesta	<code>request()</code>	<code>open()</code>	Richiedere l'uso esclusivo di un dispositivo.
Rilascio	<code>release()</code>	<code>close()</code>	Liberare il dispositivo dopo l'uso.

Se un sistema permette l'accesso incontrollato ai dispositivi da parte di utenti multipli, la contesa per l'uso può portare allo stallo (situazione in cui due o più processi si bloccano a vicenda in attesa di una risorsa).

Una volta che un dispositivo è stato richiesto e assegnato, le operazioni su di esso sono **identiche** a quelle sui file:

- `read()` : Leggere dati dal dispositivo (es. da un sensore).
- `write()` : Scrivere dati sul dispositivo (es. su una stampante).
- `reposition()` : Spostarsi all'interno del dispositivo (rilevante per dischi, non per stampanti).

Molti sistemi operativi, in particolare Unix, adottano una struttura unificata file-dispositivi. Questo significa che si usa lo stesso identico set di chiamate di sistema (`open`, `read`, `write`, `close`) sia per gestire i file su disco sia per interagire con i dispositivi

hardware (tastiera, monitor, dischi).

Il modo in cui file e dispositivi appaiono all'utente (**interfaccia utente**) può essere simile anche se le chiamate di sistema sottostanti sono diverse (questo dipende dalle scelte fatte dal progettista del sistema operativo).

Comunicazione

Ci sono due modelli principali che i processi utilizzano per comunicare e scambiare dati tra loro:

Modello a scambio di messaggi

Si basa sull'invio e la ricezione di dati incapsulati in messaggi tra i processi comunicanti.

- **Identificazione:** i processi devono conoscere l'identità del partner di comunicazione, ciò avviene tramite l'uso di nomi macchina e nomi di processo, convertiti in identificatori che il SO può usare (`get_hostid()` , `get_processid()`).
- **Connessione:** la comunicazione inizia con l'apertura di un collegamento o canale (`open_connection()`). Il processo ricevente deve acconsentire (`accept_connection()`).
- **Client-Server:** spesso, la comunicazione è gestita da programmi di sistema dedicati, detti **demoni**. Il processo che avvia la comunicazione è il client; il demone che attende la richiesta è il server (`wait_for_connection()`).
- **Scambio:** una volta stabilito il collegamento, i messaggi vengono scambiati usando chiamate dedicate (`read_message()` , `write_message()`).
- **Termine:** la comunicazione si conclude con `close_connection()` .
- **Vantaggio:** è ideale per trasferire piccole quantità di dati ed è l'unico metodo pratico per la comunicazione tra calcolatori diversi su una rete.

Modello a memoria condivisa

I processi creano e accedono a un'area di memoria comune per scambiarsi informazioni ad alta velocità.

- **Deroga alla sicurezza:** normalmente, il SO isola i processi per motivi di sicurezza (un processo non può accedere alla memoria di un altro).
- **Creazione/Accesso:** i processi devono usare chiamate di sistema specifiche (`shared_memory_create()` , `shared_memory_attach()`) per consentire la condivisione di una porzione di memoria;
- **Scambio:** una volta creata l'area condivisa, i processi leggono e scrivono i dati in quest'area **direttamente**, alla **velocità della memoria del computer**.
- **Autogestione:** il SO non ha controllo sul formato o sulla posizione dei dati all'interno della memoria condivisa.
- **Responsabilità dei processi:** i processi sono responsabili della sincronizzazione e della protezione per evitare che scrivano contemporaneamente nello stesso punto (problema di sincronizzazione).
- **Vantaggio:** offre la massima velocità e semplicità per la comunicazione all'interno dello stesso calcolatore, poiché evita il sovraccarico di impacchettare e spaccettare messaggi.

Entrambi i modelli sono diffusi e spesso coesistono nello stesso SO. La scelta dipende dal contesto.

Servizi di sistema

I servizi di sistema sono programmi che non sono parte del kernel del SO ma che offrono funzionalità utili e un ambiente più comodo per lavorare. Si classificano in diverse categorie:

- **Gestione fondamentale**
 - **Gestione dei File:** programmi per le operazioni di base su file e directory.
 - **Informazioni di stato:** utilità che chiedono e visualizzano dati sullo stato del sistema (data, ora, memoria libera, spazio su disco, utenti attivi, ecc.) o informazioni più dettagliate su debug e prestazioni. Spesso utilizzano un registro per memorizzare le configurazioni.
- **Sviluppo e esecuzione**
 - **Modifica dei file:** editor di testo e comandi speciali per la creazione e la modifica del contenuto del file.
 - **Ambienti di supporto alla programmazione:** strumenti essenziali per gli sviluppatori, come compilatori, assembleri, interpreti e debugger.
 - **Caricamento ed esecuzione dei programmi:** programmi che prendono un file eseguibile e lo preparano per l'esecuzione. Includono caricatori (**loaders**) e editor di collegamenti (**linkage editor**).
- **Interazione e sfondo**

- **Comunicazioni:** programmi che facilitano lo scambio di dati tra utenti o computer.
- **Servizi in background (demoni / servizi / sottosistemi):** programmi di sistema che si avviano con il SO e restano in esecuzione continua per svolgere compiti specifici.

Linker e Loader

Quando si crea un programma, questo viene inizialmente salvato su disco in forma di file binario eseguibile. Tuttavia, per essere eseguito su una cpu, il programma deve essere caricato in memoria e diventare parte di un processo.

Compilazione e Collegamento (Linking)

1. **Compilazione:** i file sorgenti vengono convertiti in file oggetto rilocabili. Sono "rilocabili" perché il codice è progettato per funzionare indipendentemente dalla posizione precisa che occuperà in memoria.
2. **Linking (Collegamento):** il Linker combina il file oggetto rilocabili in un singolo file binario eseguibile. In questa fase vengono incluse anche le funzioni necessarie prese dalle librerie.

Caricamento (Loading) e Rilocalizzazione (Relocation)

- **Caricamento:** il Loader è il programma che prende il file eseguibile binario e lo carica nella memoria (RAM) del computer rendendolo pronto per l'esecuzione sulla cpu.
 - Quando si digita un comando, la shell usa `fork()` per creare un nuovo processo e poi `exec()` per richiamare il loader, che carica il programma nello spazio di memoria del nuovo processo.
- **Rilocalizzazione:** è un'attività cruciale che assegna gli indirizzi finali in memoria a tutte le componenti del programma. Questo garantisce che le funzioni (incluse quelle delle librerie) e le variabili vengano richiamate agli indirizzi corretti durante l'esecuzione.

Collegamento dinamico (Dynamic Linking)

- La maggior parte dei sistemi moderni utilizza il collegamento dinamico per le librerie.
- **Vantaggio:** invece di includere tutte le librerie nel file eseguibile in fase di linking, esse vengono collegate solo in modo condizionale e caricate in memoria solo se e quando sono richieste durante l'esecuzione del programma.
- Questo approccio risparmia spazio su disco e memoria, poiché le librerie non utilizzate non vengono caricate e una singola copia della libreria dinamica può essere condivisa tra più processi.

Perché le applicazioni dipendono dal SO

La ragione fondamentale della dipendenza è che ogni SO fornisce un ambiente operativo unico e specifico al quale le applicazioni devono conformarsi.

L'ostacolo principale è che ogni SO offre un insieme univoco di chiamate di sistema.

- Le chiamate di sistema sono i servizi di base che un'applicazione richiede al SO.
- Queste chiamate differiscono enormemente tra i sistemi per:
 - nomi, operandi e ordine;
 - metodo con cui l'applicazione le invoca;
 - numerazione e significato delle chiamate stesse.

Poiché un'applicazione è scritta per invocare i servizi in un modo specifico di un SO, non funzionerà su un altro SO che non offre gli stessi servizi in quel modo.

Per far funzionare un'applicazione su più sistemi operativi, gli sviluppatori utilizzano tre approcci principali:

Approccio	Funzionamento	Prestazioni / Limiti
1. Linguaggi Interpretati	L' interprete (es. Python, Ruby) legge il codice riga per riga, traduce ogni istruzione ed esegue le chiamate di sistema native.	Inferiori (rispetto a un programma nativo). L'interprete supporta solo un sottoinsieme delle funzionalità del SO.
2. Macchina Virtuale (VM)	L'applicazione (es. Java bytecode) viene eseguita all'interno di un ambiente virtuale (es. JVM). Il Runtime Environment (RTE) della VM è l'elemento che viene "portato" sui diversi SO.	Simili agli interpreti; prestazioni spesso inferiori a quelle native e limitazioni sulle funzionalità specifiche del SO.
3. Compilazione Nativa (Porting)	Lo sviluppatore usa un linguaggio o un'API standard (es. POSIX) e ricompila l'intero codice per generare un file binario specifico per ogni SO e architettura.	Ottime prestazioni (è codice nativo), ma il porting richiede molto tempo, deve essere ripetuto per ogni nuova versione ed è costoso da mantenere e testare.

Anche se l'API (Application Programming Interface, il livello funzionale per i programmatori) fosse uniforme, l'esecuzione rimane difficile a causa delle differenze sottostanti.

- **Formato Binario Unico:** Ogni SO richiede che il file eseguibile (il binario) abbia un formato specifico (es. **ELF** per Linux/Unix, **PE** per Windows). Questo formato definisce dove devono trovarsi l'intestazione, le istruzioni e le variabili affinché il SO possa caricarle.
- **Set di Istruzioni CPU:** Le diverse architetture hardware (es. Intel x86, ARMv8) hanno **set di istruzioni** differenti. Un binario compilato per una CPU non può essere eseguito su un'altra.
- **ABI (Application Binary Interface):** L'ABI definisce i dettagli di **bassissimo livello** su come il codice binario deve interfacciarsi con il SO su una data architettura (es. dimensione degli indirizzi, come vengono passati i parametri alle chiamate di sistema, l'organizzazione dello stack). Un'ABI è **specificata per una coppia SO + Architettura** e quindi fa poco per la compatibilità multiplatforma.

In sintesi, affinché un programma funzioni, il suo codice deve essere **scritto o compilato** per aderire sia alle API (funzionalità di alto livello) sia all'ABI (dettagli di basso livello) dello specifico sistema operativo e della specifica CPU su cui deve essere eseguito.